LAPORAN PROGRES TUGAS BESAR CLOUD COMPUTING

Pertemuan 2: Environment Setup & Core Deployment

Mata Kuliah: Cloud Computing (3 SKS) • 29 Oktober 2026

Topik Proyek: **Auto-Scaling & Load Balancing Sistem War Tiket Menggunakan Docker Swarm**

Studi Kasus: Sistem War Tiket Konser Musik (Flash Crowd) **Repositori Git:** github.com/Cahyooo69/Auto-Scaling-Load-Balancing

Kelompok: **Kelompok 1** — *Anggota Tim:*

- **Muhammad Ilham Nurcahyo** (09040623063)
- **Muhammad Daud Rizaldiansyah** (09040623061)
- **Hafid Aji Fauzan** (0920623027)
- **Davin Aryaguna Hermansyah** (09020623020)

1. PENDAHULUAN & TARGET LUARAN PERTEMUAN 2

Berdasarkan silabus dan timeline mata kuliah Cloud Computing (3 SKS), sasaran utama untuk **Pertemuan 2 (Environment Setup & Core Deployment)** adalah:

> **Luaran Wajib Pertemuan 2:**
> *"Seluruh container dapat berjalan (up & running) dalam jaringan internal Docker dengan resource limit yang sudah diterapkan."*

Kelompok 1 telah berhasil menyelesaikan seluruh deployment stack bernama `wartiket` pada klaster Docker Swarm lokal. Laporan ini menyajikan bukti operasional seluruh container yang aktif, verifikasi pembatasan sumber daya (*resource limits*) di level kernel, dan validasi komunikasi jaringan internal.

2. BUKTI OPERASIONAL SELURUH CONTAINER (UP & RUNNING)

Semua service yang didefinisikan pada berkas `docker-compose.yml` telah berhasil di-deploy ke klaster Docker Swarm dengan status **100% Running** tanpa error:

Service Name	Container ID	Status Desired	Current State	Port Binding	Fungsi Arsitektur
`wartiket_nginx`	`i1orif57jdon`	Running	Running (Healthy)	`*:80->80/tcp`	Reverse Proxy & Ingress Load Balancer (Port publik gerbang masuk).
`wartiket_app.1`	`ri2ub2cbty8t`	Running	Running (Healthy)	`3000/tcp` (Internal)	Replika Worker Backend 1 (Mock API checkout tiket dengan delay 50-100ms).
`wartiket_app.2`	`w06pvpk878z9`	Running	Running (Healthy)	`3000/tcp` (Internal)	Replika Worker Backend 2 (Menyeimbangkan beban transaksi bersama replika 1).
`wartiket_redis`	`wvtwrcqh4wks`	Running	Running (Healthy)	`6379/tcp` (Internal)	In-Memory Datastore & Atomic Lua Script Locking (Stok tiket persistent).

```
$ docker stack services wartiket
ID             NAME             MODE         REPLICAS   IMAGE                  PORTS
lbuec3bv49sx   wartiket_app     replicated   2/2        wartiket-app:latest
x7zlrm1hedl3   wartiket_nginx   replicated   1/1        wartiket-nginx:latest  *:80->80/tcp
50k0j53unes3   wartiket_redis   replicated   1/1        redis:7-alpine
```

3. VERIFIKASI PENERAPAN RESOURCE LIMITS

Ketentuan standar tugas besar 3 SKS mewajibkan pembatasan sumber daya ketat pada setiap container:

- **Maksimal CPU:** `0.25 core` (maksimal 25% dari 1 core CPU host).
- **Maksimal Memori RAM:** `256 MB` (268.435.456 Bytes).
- **Reservasi Sumber Daya:** `0.10 core CPU` dan `64 MB RAM`.

Penerapan batasan ini diverifikasi langsung dari Docker Daemon menggunakan perintah inspeksi metadata tugas Swarm:

Service	Spesifikasi Limit (Compose)	Nilai Inspeksi Kernel (Docker Inspect)	Status Kepatuhan
Backend API (`wartiket_app`)	CPU: 0.25 | RAM: 256M	`NanoCPUs: 250000000 | MemoryBytes: 268435456`	**Memenuhi Standar**
Load Balancer (`wartiket_nginx`)	CPU: 0.25 | RAM: 256M	`NanoCPUs: 250000000 | MemoryBytes: 268435456`	**Memenuhi Standar**
Database Redis (`wartiket_redis`)	CPU: 0.25 | RAM: 256M	`NanoCPUs: 250000000 | MemoryBytes: 268435456`	**Memenuhi Standar**

Bukti Eksekusi Terminal (Docker Inspect):

```
$ docker service inspect wartiket_app --format "{{json .Spec.TaskTemplate.Resources}}"
{"Limits":{"NanoCPUs":250000000,"MemoryBytes":268435456},"Reservations":{"NanoCPUs":100000000,"MemoryBytes":6
7108864}}
```

Keterangan: Nilai 250.000.000 NanoCPUs setara tepat dengan 0.25 CPU Core, dan 268.435.456 Bytes setara tepat dengan 256 MB RAM.

4. SKEMA JARINGAN INTERNAL & UJI KONEKTIVITAS

Seluruh container terhubung di dalam jaringan internal bertipe **Docker Overlay Network** dengan konfigurasi berikut:

- **Nama Jaringan:** `wartiket_wartiket-net` (Driver: `overlay`, Scope: `swarm`).
- **Alokasi Subnet:** `10.0.1.0/24` dengan gateway internal otomatis.
- **Isolasi Port:** Port backend (3000) dan port database (6379) tidak dipublikasikan ke host, sehingga hanya dapat diakses melalui jaringan internal oleh Nginx.
- **Service Discovery & VIP:** Nginx me-resolve nama host `app:3000` melalui DNS internal Docker Swarm (`127.0.0.11`).

```
$ docker network inspect wartiket_wartiket-net --format "Driver: {{.Driver}} | Scope: {{.Scope}} | Subnet: {{rang
e .IPAM.Config}}{{.Subnet}}{{end}}"
Driver: overlay | Scope: swarm | Subnet: 10.0.1.0/24
```

5. PENGUJIAN FUNGSIONAL END-TO-END JARINGAN

Untuk membuktikan bahwa seluruh container tidak hanya berjalan tetapi juga saling terhubung secara fungsional (*end-to-end*), telah dilakukan pengujian transaksi bersamaan melalui port 80 Nginx menggunakan skrip penguji:

```
$ node test-quick.js
Sending 20 concurrent checkout requests to http://localhost/tickets/buy...
┌─────────┬────┬────────┬─────────┬──────────────────┬─────────────┬─────────┬──────────────────────────────┐
│ (index) │ ID │ Status │ Latency │ ServedBy         │ Remaining   │ Success │ Info                         │
├─────────┼────┼────────┼─────────┼──────────────────┼─────────────┼─────────┼──────────────────────────────┤
│ 0       │ 1  │ 200    │ '115ms' │ '86d0051a2950'   │ 999         │ true    │ 'Tiket berhasil didapatkan!' │
│ 1       │ 2  │ 200    │ '90ms'  │ '83f7bdab3528'   │ 998         │ true    │ 'Tiket berhasil didapatkan!' │
│ ...     │ .. │ 200    │ ...     │ ...              │ ...         │ true    │ 'Tiket berhasil didapatkan!' │
│ 19      │ 20 │ 200    │ '98ms'  │ '86d0051a2950'   │ 980         │ true    │ 'Tiket berhasil didapatkan!' │
└─────────┴────┴────────┴─────────┴──────────────────┴─────────────┴─────────┴──────────────────────────────┘

--- Load Balancing Distribution Across Containers ---
┌──────────────────┬────────┐
│ (index)          │ Values │
├──────────────────┼────────┤
│ 86d0051a2950     │ 10     │
│ 83f7bdab3528     │ 10     │
└──────────────────┴────────┘
```

Kesimpulan Hasil Pengujian Fungsional:

1. **Konektivitas Nginx → Backend:** Berhasil 100%. Request dari port 80 diteruskan dan dibagi seimbang 50:50 (10 request ke Worker 1 dan 10 request ke Worker 2).
2. **Konektivitas Backend → Redis:** Berhasil 100%. Transaksi checkout berhasil mengeksekusi script Lua atomic decrement di Redis, terbukti dari sisa stok yang tereduksi konsisten dari 999 ke 980.
3. **Target SLA:** Seluruh respon memiliki latency rata-rata ~95ms, jauh di bawah batas maksimum SLA yaitu 200ms.

6. PEMENUHAN TARGET LUARAN PERTEMUAN 2

Indikator Keberhasilan Pertemuan 2	Target Persyaratan	Hasil Verifikasi Lapangan	Status Capaian
Status Container	Semua container up & running	4 container aktif (Nginx, App.1, App.2, Redis) berstatus *Running*.	**Selesai 100%**
Isolasi Jaringan	Jaringan internal Docker	Docker Overlay Network (`10.0.1.0/24`) dengan proteksi port internal.	**Selesai 100%**
Penerapan Resource Limit	Maksimal 0.25 CPU & 256MB RAM	Terverifikasi pada Docker Inspect (`250000000` NanoCPUs & `256M`).	**Selesai 100%**
End-to-End Core Deployment	Layanan terintegrasi & responsif	Uji 20 transaksi sukses 100% dengan round-robin load balancing.	**Selesai 100%**